Chapter 3 · Lesson 1 — Anatomy of a Component

Chapter 3 · Lesson 1 — Anatomy of a Component

> *Goal: by the end of this lesson, you can look at any React component file and identify every part of it — the function, the props, the JSX, the return, the imports — and explain what each one does to a junior developer.*

What is a "component", really?

In React, a component is **just a function**. Specifically, a function that:

1. Takes a single argument called `props` (an object with whatever data the parent passes in).
2. Returns either a piece of JSX (looks like HTML), a string, a number, an array, or `null`.

That's it. There's no class, no constructor, no `extends`, no decorators. **A component is a function.** If you can write a function, you can write a React component.

Here's the smallest possible React component:

```
function Hello() {
  return <h1>Hello, world.</h1>;
}
```

Six tokens. One return. That's a complete, valid, renderable React component. You could put it on a page right now.

Functional vs class components — the 2026 answer

If you've read any React tutorial older than 2020, you've probably seen `class MyComponent extends React.Component { render() { ... } }`. That's the **class**

component style. It still works, but **you should never write a new class component in 2026.**

The React team has explicitly said functional components + hooks are the path forward. Class components are a legacy style that exists for backwards compatibility. Every new component in this course is a function.

If an interviewer asks you about class components, the answer is: *"I know they exist, I can read them, I'd refactor to functional + hooks if I owned the code."* That's enough.

What is JSX?

The thing that looks like HTML inside a component is **JSX** — JavaScript XML. It's a syntax extension to JavaScript that lets you write tag-like markup directly in your code.

```
return <h1>Hello, world.</h1>;
```

That `<h1>...</h1>` is JSX. It is **not HTML**. It looks like HTML, but it's actually compiled into a JavaScript function call. Specifically, a Next.js / React build pipeline transforms the line above into:

```
return React.createElement('h1', null, 'Hello, world.');
```

`React.createElement(type, props, ...children)` is what JSX is "really" doing under the hood. The compiler does this for you so you can write the more readable JSX form. **Knowing this saves you from a dozen confusing error messages later.**

Why does this matter?

Three implications fall out of "JSX is just function calls":

1. JSX returns a value. Because it compiles to `React.createElement(...)`, JSX produces a **JavaScript object** (a "React element") at runtime. You can assign it to a variable:

```
const heading = <h1>Hello</h1>;   // heading is an object, not HTML
return <div>{heading}</div>;       // and you can embed it inside more JSX
```

That's why you can store JSX in a variable, return it from a function, pass it as a prop, etc. **JSX is data, not markup.**

2. You can use JavaScript inside JSX with `{}`. Curly braces inside JSX let you embed any JavaScript expression:

```
const name = 'Wu Yize';
return <h1>Hello, {name}!</h1>;          // "Hello, Wu Yize!"
return <p>{1 + 2}</p>;                    // "3"
return <p>{teams.map(t => <span key={t.name}>{t.name}</span>)}</p>;  // a li
```

Anywhere a JS expression is valid, `{}` lets you drop one in. Statements (like `if` and `for`) are not expressions, so they don't fit; you'll see ternaries (`condition ? a : b`) and `.map()` calls instead.

3. Naming and capitalization matter. JSX distinguishes between **tags** and **components** by the first letter:

```
<h1>...</h1>      // lowercase → HTML element
<Hello />         // uppercase → React component (your function)
```

Compiled output:

```
React.createElement('h1', null, ...)    // string type → DOM element
React.createElement(Hello, null)        // function type → React component
```

If you accidentally name your component `function hello()` instead of `function Hello()`, JSX will treat `<hello />` as the HTML tag `<hello>` (which doesn't exist) and your component will silently not render. **Always capitalize component names.**

The full anatomy: a real component, line by line

Open `components/standings/StatCard.tsx` from the repo. It's a small but complete component:

```
import type { LucideIcon } from 'lucide-react';

interface StatCardProps {
  icon: LucideIcon;
  label: string;
  value: string | number;
  color: string;
```

```
  bgColor: string;
}

export default function StatCard({ icon: Icon, label, value, color, bgColor
  return (
    <div
      style={{
        background: `linear-gradient(135deg, ${bgColor} 0%, ${bgColor}DD 100%
        color: '#FFFFFF',
        padding: 24,
        borderRadius: 16,
        boxShadow: `0 8px 24px ${bgColor}40`,
      }}
    >
      <Icon size={28} strokeWidth={2.5} />
      <div style={{ fontSize: 13, letterSpacing: '1.5px', fontWeight: 600, op
        {label}
      </div>
      <div style={{ fontSize: 32, fontWeight: 900, fontFamily: "'Roboto Slab
        {value}
      </div>
    </div>
  );
}
```

Let's dissect it from top to bottom.

Line 1: imports

```
import type { LucideIcon } from 'lucide-react';
```

This component imports a TypeScript **type** (not a value) from the `lucide-react` package. `LucideIcon` is the type of any icon component (Trophy, Calendar, Star, etc.). The actual icon will be passed in as a prop — this file doesn't pick a specific one.

`import type` (vs plain `import`) tells TypeScript "this is type-only; erase from runtime output." Senior projects use it consistently.

Lines 3–9: the props interface

```
interface StatCardProps {
  icon: LucideIcon;
  label: string;
  value: string | number;
  color: string;
  bgColor: string;
}
```

This is the **props contract** for the component. Five fields:

- `icon: LucideIcon` — the Lucide icon component to render (e.g. `Trophy`, `Star`).
- `label: string` — the text under the icon ("LEADER", "MATCHES", etc.).
- `value: string | number` — the big number/text in the middle. **Union type** — accepts either.
- `color: string` — primary accent color (used in some variants).
- `bgColor: string` — the gradient background base.

By declaring this interface, **the component documents its API**. Any caller using this component must pass props matching this shape, or TypeScript yells at them. Conversely, the *inside* of the component knows exactly what it has to work with.

The convention `<ComponentName>Props` for the interface name is universal in the React community. Stick with it.

Line 11: the function declaration

```
export default function StatCard({ icon: Icon, label, value, color, bgColor
```

Several things going on here:

`export default` Means *"this is the main export of the file, and a caller can `import StatCard from './StatCard'` without curly braces."*

Compare to `export function StatCard()` (a "named export") which would require `import { StatCard } from './StatCard'` with braces.

Convention in this codebase (and most React projects): **one component per file, exported**

as default. The reason is consistency with import syntax — every file you import a component from looks the same. There are religious wars about default vs named exports; pick one and don't mix.

```
function StatCard({ icon: Icon, label, value, color, bgColor }: StatCardProps
```

Destructuring the props This is JavaScript object destructuring with a TypeScript annotation. It's equivalent to:

```
function StatCard(props: StatCardProps) {
  const Icon = props.icon;
  const label = props.label;
  // ...
}
```

But shorter and more idiomatic.

The `icon: Icon` part is a **rename** — we're pulling out the prop called `icon` and giving it the local name `Icon` (capitalized). Why? Because we're going to render it as a JSX tag (`<Icon ... />`), and JSX needs uppercase to recognize it as a component. **Without the rename, `<icon />` would compile to a non-existent HTML element.**

This is a small but classic React quirk. Whenever you accept a component as a prop, capitalize it locally.

Lines 12–32: the return — JSX

```
return (
  <div style={{ ... }}>
    <Icon size={28} strokeWidth={2.5} />
    <div style={{ ... }}>{label}</div>
    <div style={{ ... }}>{value}</div>
  </div>
);
```

Three children of a `<div>`:

1. The icon component (e.g. `<Trophy size={28} />`).

2. A `<div>` containing the label text.
3. A `<div>` containing the value (number or string).

Notice the **`return ( ... )`** with parens around the JSX. Why?

When JSX spans multiple lines, JavaScript's automatic semicolon insertion (ASI) can confuse the parser. Wrapping the JSX in `( )` makes it unambiguous that this is one big expression. **Always wrap multi-line JSX in parens.** It's not strictly required by the language; it's required by every real codebase to avoid ASI bugs.

The inline style attribute

```
<div
  style={{
    background: `linear-gradient(135deg, ${bgColor} 0%, ${bgColor}DD 100%)`,
    color: '#FFFFFF',
    padding: 24,
    borderRadius: 16,
    boxShadow: `0 8px 24px ${bgColor}40`,
  }}
>
```

Three things happening:

Two layers of `{}` `style={{...}}` has TWO layers of braces. The outer pair is JSX's "embed an expression here." The inner pair is a JavaScript **object literal**. So it's actually `style={` + `{...}` + `}` — *the JSX expression `{...}` evaluates to an object literal that React assigns to the style attribute.*

Beginners often forget the second pair and write `style={...}` — that's a syntax error.

CamelCase property names CSS uses kebab-case (`background-color`, `border-radius`). React inline styles use camelCase (`backgroundColor`, `borderRadius`). Why? Because JavaScript object keys can't have hyphens unless quoted, and unquoted keys are nicer to type.

Internally React converts `backgroundColor: 'red'` to `style.backgroundColor = 'red'` on the DOM element. The browser handles the rest.

```
padding: 24,           // OK — React adds 'px' for you
fontSize: 14,          // OK — same
zIndex: 50,            // OK — but no 'px' added (zIndex is unitless)
padding: '24px',       // also OK
padding: '14px 16px',  // string when you need multiple values
```

Numbers vs strings For most CSS properties that accept a length, React assumes pixels if you pass a number. Pass a string when you need a unit other than `px` (`'2rem'`, `'50%'`) or multiple values.

Template literals for dynamic colors

```
background: `linear-gradient(135deg, ${bgColor} 0%, ${bgColor}DD 100%)`,
```

This uses a JS **template literal** (backticks + `${...}`). The expression `${bgColor}` is the value of the `bgColor` prop interpolated into the string.

The trailing `DD` is hex shorthand for ~87% opacity (`DD` = 221 / 255). So if `bgColor = '#0F5132'`, the gradient runs from solid `#0F5132` to `#0F5132DD` — same color, slightly transparent. This produces a smooth 1-color gradient that doesn't look flat.

This kind of **dynamic styling based on props** is exactly why we use inline styles instead of Tailwind classes. There's no Tailwind class for "this color, 87% opacity, with a 135° gradient" computed at runtime. Inline wins.

Self-closing tags

```
<Icon size={28} strokeWidth={2.5} />
```

Components with no children use **self-closing syntax**. Same rule as `<img />` and `<br />` in XHTML. A non-self-closing version like `<Icon size={28}></Icon>` would also work, but self-closing is shorter and more conventional for empty elements.

Recap: the four parts of every component file

You'll see this skeleton everywhere:

```
import { ... } from '...';   // 1. imports

interface FooProps {           // 2. props contract (TypeScript)
  bar: string;
}

export default function Foo({ bar }: FooProps) {   // 3. function with destr
  return (                                          // 4. return JSX
    <div>{bar}</div>
  );
}
```

That's it. **Every component you'll ever write follows this pattern.** Once you see it three times you'll see it everywhere.

Server vs client components — a teaser

You may have noticed: at the top of `components/standings/StatCard.tsx` there's no `'use client'` directive. Neither was there one on `app/layout.tsx`. So what kind of component is this — server or client?

Default is server in the App Router. A component is a server component unless something inside it (or its parent) has `'use client'` declared.

`StatCard` is a server component. So is `app/layout.tsx`. They render to HTML on the server, ship zero JavaScript to the browser. That's fine because they don't use any browser-only features.

`<SnookerFantasyLeague>` (the orchestrator) WILL need `'use client'` because it uses `useState`. Once a parent is marked client, all its descendants are also rendered client-side. **You only need `'use client'` at the boundary** — the highest component in your tree that needs interactivity. Everything below that boundary is automatically client.

Chapter 4 will introduce `'use client'` formally. For now, just know:

- Server components are the default.
- They're free (zero JS shipped to the browser).
- They cannot use `useState`, `useEffect`, `onClick`, `localStorage`, or any browser API.

- `'use client'` opts a file (and its descendants) into client rendering.

Vibe prompt you would have used

> *"Write me a `StatCard` component in `components/standings/StatCard.tsx`. Props: `icon: LucideIcon`, `label: string`, `value: string | number`, `color: string`, `bgColor: string`. Render a div with a gradient background using `bgColor` (linear-gradient 135deg from full to ~87% opacity), white text, 24px padding, 16px border radius, big shadow tinted with bgColor. Inside: the icon at 28px stroke 2.5, then the label in small letterspaced text, then the value as a big serif heading. Inline styles only. TypeScript, default export."*

The trick is naming **every prop, every style detail, every visual choice.** When the spec is this specific, the LLM has nothing to invent. **Specificity in, predictability out.**

CHECK YOURSELF

1. **Why does the prop destructuring use `{ icon: Icon }` instead of just `{ icon }`?** What would break if you used the lowercase form?
2. **`style={{ ... }}` has two layers of braces. Walk through what each layer does.** What error would you get if you wrote `style={ ... }` (one layer)?
3. **`padding: 24` works, but `font-size: '14'` does not. Why?** What's the difference between unitless numbers and string values in React style objects?
4. **A teammate writes `<icon size={28} />` and gets a console warning about an unknown HTML element. What's wrong, and what's the one-character fix?**
5. **`StatCard.tsx` doesn't have `'use client'` at the top. Is it a server component or a client component? Why?** Could you `useState` inside it as-is? Could a parent component with `'use client'` use `<StatCard />` inside it?

When you've answered these, head to **02-rendering-the-standings.md** — we build our first real, data-driven component.